

```

#include <iostream>

#include <vector>

#include <queue>

#include <limits>

#include <utility>

using namespace std;

// ----- Print Path -----

void printPath(vector<int> &parent, int node) {
    if (node == -1)
        return;
    printPath(parent, parent[node]);
    cout << node;
    if (parent[node] != -1)
        cout << " -> ";
}

// ----- Dijkstra Algorithm -----

void dijkstra(vector<vector<pair<int,int>>> &graph,int start) {
    int n = graph.size();
    vector<int> distances(n, numeric_limits<int>::max());
    vector<int> parent(n, -1);
    distances[start] = 0;

    // Min Heap -> (distance, node)
    priority_queue< pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
    pq.push({0, start});
    int step = 1;
    while (!pq.empty()) {
        cout << "\n--- Step " << step << " ---\n";
    }
}

```

```

// ----- Print Priority Queue -----
cout << "Priority Queue: ";

priority_queue< pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> temp = pq;

while (!temp.empty()) {
    cout << "(" << temp.top().first << ", " << temp.top().second << ") ";
    temp.pop();
}

cout << endl;

// ----- Print Distances -----
cout << "Distances: ";

for (int i = 0; i < n; i++) {
    if (distances[i] == numeric_limits<int>::max())
        cout << "inf ";
    else
        cout << distances[i] << " ";
}

cout << endl;

int current_distance = pq.top().first;
int current_node = pq.top().second;
pq.pop();

cout << "Selected Node: " << current_node << " with distance " << current_distance << endl;

// Skip if better path exists
if (current_distance > distances[current_node]) {
    cout << "Skipped " << "(better path already exists)\n";
    continue;
}

```

```

// ----- Explore Neighbors -----
for (auto &edge : graph[current_node]) {
    int neighbor = edge.first;
    int weight = edge.second;
    int new_distance = current_distance + weight;

    cout << "Checking edge " << current_node << " -> " << neighbor << " with weight " << weight
<< endl;

    if (new_distance < distances[neighbor]) {
        cout << "Updating distance of node " << neighbor << " from ";
        if (distances[neighbor] == numeric_limits<int>::max())
            cout << "inf";
        else
            cout << distances[neighbor];

        cout << " to " << new_distance << endl;
        distances[neighbor] = new_distance;

        // Store Parent
        parent[neighbor] = current_node;

        pq.push({ new_distance, neighbor});
    }

    else {
        cout << "No update needed for node " << neighbor << endl;
    }
}
step++;
}

```

```

// ----- Final Result -----

cout << "\nFinal Shortest Distances:\n";

for (int i = 0; i < n; i++) {
    cout << i << " -> ";

    if (distances[i] == numeric_limits<int>::max())
        cout << "inf";

    else
        cout << distances[i];

    cout << endl;
}

// ----- Print Shortest Paths -----

cout << "\nShortest Paths:\n";

for (int i = 0; i < n; i++) {
    cout << "Path to node " << i << ": ";

    // ----- Disconnected Graph Check -----

    if (distances[i] == numeric_limits<int>::max()) {
        cout << "No Path Exists";
    }

    else {
        printPath(parent, i);

        cout << " (Cost = " << distances[i] << ")";
    }

    cout << endl;
}
}

// ----- MAIN -----

int main() {

```

```

int n, e;

cout << "Enter number of vertices: ";
cin >> n;

cout << "Enter number of edges: ";
cin >> e;

vector<vector<pair<int,int>>> graph(n);

cout << "Enter edges (u v w):\n";

for (int i = 0; i < e; i++) {

    int u, v, w;

    cin >> u >> v >> w;

    // ----- Invalid Vertex Check -----
    if (u < 0 || u >= n || v < 0 || v >= n) {

        cout << "\nError: Invalid vertex number!\n";

        cout << "Vertices must be between 0 and " << n - 1 << endl;

        return 0;
    }

    // ----- Negative Weight Check -----
    if (w < 0) {

```

```

        cout << "\nError: Negative weights are not allowed " << "in Dijkstra Algorithm!\n";
        return 0;
    }

    graph[u].push_back({v, w});
    graph[v].push_back({u, w});
    // Remove above line for directed graph
}

int start;
cout << "Enter source node: ";
cin >> start;

// ----- Source Validation -----
if (start < 0 || start >= n) {
    cout << "\nError: Invalid source node!\n";
    return 0;
}

// Run Dijkstra
dijkstra(graph, start);
return 0;
}

```